CURSO ITF DE TESTING MANUAL

Proyecto final

Testing del módulo Employee de la aplicación OrangeHRM

Resumen del proyecto

Estudiante	Valentina Hatos
Rol en el proyecto	QA Engineer
Aplicación bajo prueba	OrangeHRM (opensource-demo.orangehrmlive.com)
Módulo probado	Employee – Añadir / Editar / Eliminar
Herramientas	JIRA, Zephyr Squad, Postman, MySQL Workbench
Fecha	Mayo de 2022

Índice

1. Introducción....3
1.1 Aplicación bajo prueba....3
1.2 Herramientas utilizadas....3
2. Especificaciones funcionales....3
3. Planificación de las pruebas....5
3.1 Roles y personas asignadas....5
3.2 Criterios de entrada....5
3.3 Criterios de salida....5
3.4 Alcance de las pruebas....5
3.5 Riesgos....5
3.6 Evaluación de los criterios de entrada....6
4. Monitorización y control de las pruebas....6
5. Análisis de las pruebas....7
6. Diseño de las pruebas....7
6.1 Técnicas de diseño de pruebas....7
6.2 Checklist....8
6.3 Casos de prueba....8
6.4 Checklist de pruebas API....9
7. Implementación de las pruebas....9
8. Ejecución de las pruebas....9
8.1 Ejecución de las pruebas funcionales....9
8.2 Bugs reportados....10
8.3 Pruebas API....10
9. Cierre de las pruebas....11
9.1 Evaluación de los criterios de salida....12
9.2 Conclusión....12
10. Sección SQL....12
10.1 Creación de la base de datos y las tablas....13
10.2 Inserción de datos....13
10.3 Consultas....13
11. Reflexiones finales....14

1. Introducción

Este documento describe mi proyecto final para el curso ITF de Testing Manual. El objetivo del proyecto fue aplicar los conocimientos adquiridos durante el curso en una aplicación real.

Probé el módulo Employee de OrangeHRM, una aplicación de recursos humanos de código abierto. El módulo permite al usuario añadir, editar y eliminar empleados.

El proyecto tiene dos secciones:

- **Sección de testing** – seguí las fases principales del proceso de pruebas: planificación, monitorización y control, análisis, diseño, implementación, ejecución y cierre.
- **Sección SQL** – creé una pequeña base de datos y escribí consultas para seleccionar, actualizar y eliminar datos.

1.1 Aplicación bajo prueba

Elemento	Detalles
Aplicación	OrangeHRM (demo de código abierto)
URL	https://opensource-demo.orangehrmlive.com/
Documentación API	https://orangehrm.github.io/orangehrm-api-doc/
Cuenta de prueba	Usuario: Admin | Contraseña: admin123
Módulo probado	Employee (PIM >> Add Employee, Personal Details, Employee List)

1.2 Herramientas utilizadas

Herramienta	Cómo la utilicé
JIRA	Creé el proyecto, la user story, los tests y los informes de bugs.
Zephyr Squad	Escribí los pasos de los tests, creé el ciclo de pruebas, ejecuté los tests y generé los informes.
Postman	Creé una colección de peticiones y probé la API.
MySQL Workbench	Creé la base de datos y las tablas, inserté datos y ejecuté las consultas.
Better PDF Exporter for Jira	Exporté los casos de prueba, las ejecuciones y los bugs a archivos PDF.

2. Especificaciones funcionales

Las especificaciones funcionales se redactaron como una user story en JIRA. La story es VFP-2: “As a user I want to be able to ADD/EDIT/DELETE an employee” (como usuario, quiero poder añadir, editar y eliminar un empleado). Pertenece al epic VFP-1 (EMPLOYEE module).

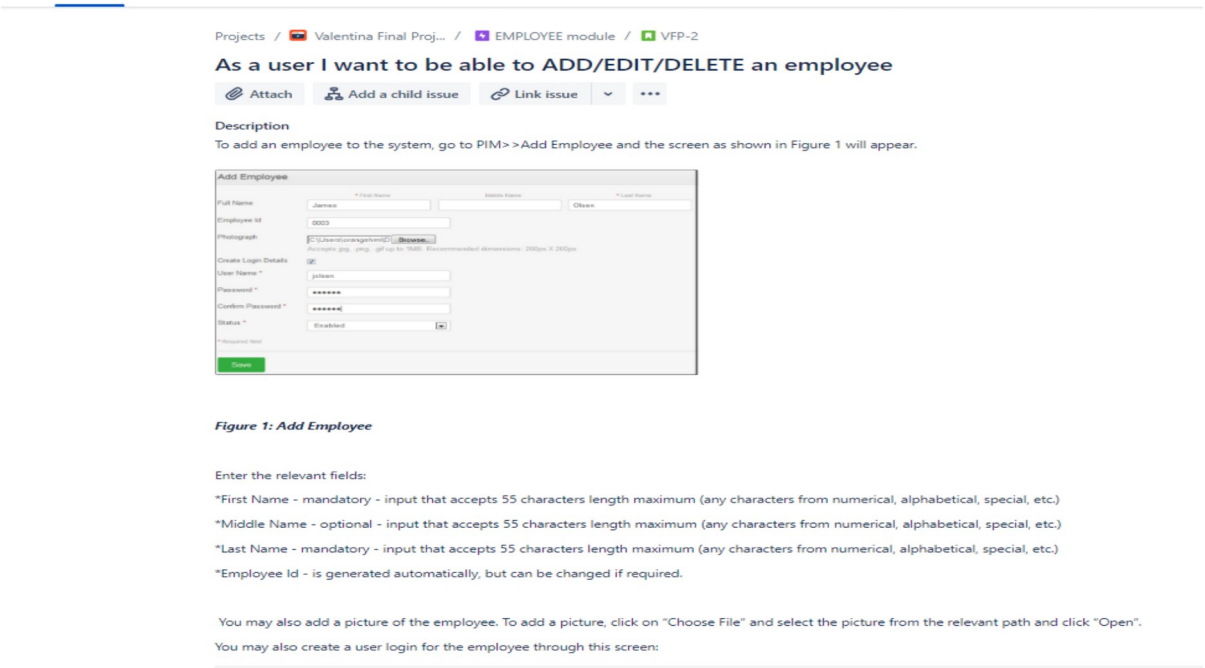


Figura 1: User story VFP-2 en JIRA

La story describe tres acciones principales:

- **Añadir un empleado** – desde PIM >> Add Employee. El usuario introduce el nombre y el Employee Id, y puede subir una foto. También puede crear datos de acceso para el empleado.
- **Editar un empleado** – desde la pantalla Personal Details. El usuario hace clic en Edit, modifica los campos y hace clic en Save.
- **Eliminar un empleado** – desde PIM >> Employee List. El usuario selecciona uno o varios empleados y hace clic en Delete.

A continuación se muestran las reglas principales de los campos.

Campo	Sección	Regla
First Name	Add / Edit	Obligatorio, máximo 55 caracteres (de cualquier tipo)
Middle Name	Add / Edit	Opcional, máximo 55 caracteres (de cualquier tipo)
Last Name	Add / Edit	Obligatorio, máximo 55 caracteres (de cualquier tipo)
Employee Id	Add / Edit	Se genera automáticamente, se puede modificar
Photograph	Add	.jpg, .png o .gif, máximo 1 MB
User Name	Login details	Obligatorio
Password	Login details	Obligatorio, se recomienda una combinación segura
Confirm Password	Login details	Obligatorio
Status	Login details	Desplegable obligatorio: Enabled o Disabled
Nationality	Edit	Se selecciona de una lista predefinida
Date of Birth	Edit	Opcional, selector de fecha, formato yyyy-mm-dd
Marital Status	Edit	Desplegable opcional: Single, Married u Other
Smoker	Edit	Casilla de verificación
Gender	Edit	Botones de opción: Male o Female
License Expiry Date	Edit	Opcional, selector de fecha, formato yyyy-mm-dd

Campo	Sección	Regla
Nick Name, Other Id, Driver's License Number, Military Service	Edit	Campos de texto opcionales

3. Planificación de las pruebas

En esta fase, elaboré el plan de pruebas. El plan describe qué se va a probar, cómo, quién lo hará y qué riesgos existen.

3.1 Roles y personas asignadas

Rol	Persona
Project Manager	Maria Macavei
Product Owner	David Dafinescu
Software Developer	Adrian Adagiu
QA Engineer	Valentina Hatos

3.2 Criterios de entrada

Las pruebas solo podían empezar cuando se cumplieran estas condiciones:

- Las especificaciones funcionales están definidas.
- Los roles necesarios para el proyecto están asignados.
- Los riesgos iniciales del proyecto se han identificado y mitigado.

3.3 Criterios de salida

Las pruebas se podían considerar terminadas cuando se cumplieran estas condiciones:

- El número de bugs sin resolver es insignificante o tienen prioridad baja.
- Se han ejecutado todos los tests.
- Todos los bugs resueltos se han vuelto a probar y han sido aprobados por el equipo de QA.
- Se ha alcanzado la fecha límite.
- No queda ningún riesgo importante sin mitigar.
- Se realiza un test de regresión exploratorio en el módulo PIM, que incluye la sección Employee.

3.4 Alcance de las pruebas

Dentro del alcance: todas las funcionalidades del módulo Employee definidas en las especificaciones. Planifiqué pruebas funcionales, pruebas de interfaz gráfica (GUI) y pruebas de API.

Fuera del alcance: pruebas de rendimiento, integración del módulo Employee con otros módulos y pruebas de compatibilidad en varios navegadores.

3.5 Riesgos

Tipo	Riesgo
Riesgos del proyecto	Falta de experiencia del equipo de QA; el periodo de prueba corto de

Tipo	Riesgo
	Zephyr Squad; el entorno de pruebas podría no estar disponible.
Riesgos del producto	Las reglas de validación de los campos podrían ser demasiado restrictivas para el usuario final.

3.6 Evaluación de los criterios de entrada

Revisé los criterios de entrada antes de empezar. Se cumplían todos, así que el proceso de pruebas continuó.

4. Monitorización y control de las pruebas

Durante las pruebas, generé informes periódicos en Zephyr. Los informes mostraban el estado actual del proceso de pruebas. Si aparecía un problema importante, se podían tomar medidas de control.

El 19 de mayo de 2022 ya se había ejecutado más del 50% de los casos de prueba. En ese momento, generé el informe de estado que se muestra a continuación. El gráfico Daily Test Execution Progress mostraba 21 ejecuciones superadas y 2 fallidas.

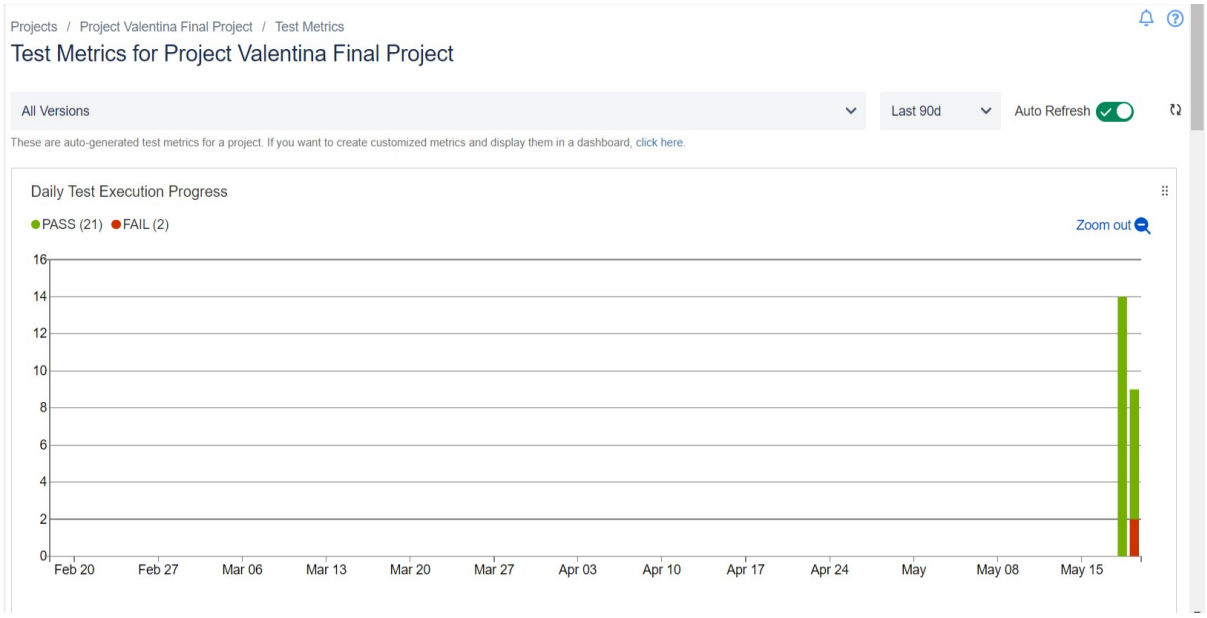


Figura 2: Métricas de prueba el 19 de mayo de 2022

También revisé la matriz de trazabilidad en Zephyr. Mostraba que los 44 tests estaban vinculados a la story VFP-2 y que, hasta ese momento, se habían reportado 2 defectos.

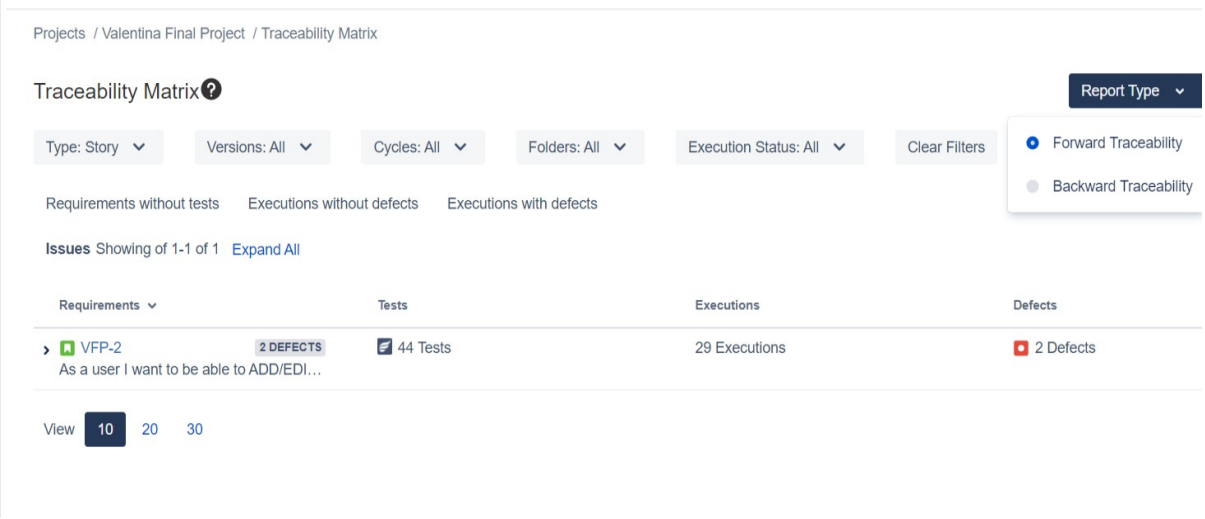


Figura 3: Matriz de trazabilidad el 19 de mayo de 2022

El proceso de pruebas avanzaba según lo previsto, así que continué con el resto de los casos de prueba.

5. Análisis de las pruebas

En esta fase, analicé las especificaciones para saber qué había que probar. Identifiqué las siguientes condiciones de prueba:

- Introducir datos solo en los campos obligatorios y comprobar que el empleado se crea/actualiza.
- Introducir datos en todos los campos disponibles y comprobar que el empleado se crea/actualiza.
- Dejar vacíos los campos obligatorios y comprobar que el empleado no se puede crear/actualizar.
- Comprobar que se puede eliminar un empleado.
- Ver los datos del empleado y comprobar que son correctos.
- Ver todos los empleados en una lista.
- Comprobar todas las reglas de validación de los campos.

6. Diseño de las pruebas

6.1 Técnicas de diseño de pruebas

Basándome en las especificaciones, utilicé tres técnicas de diseño de pruebas:

- **Partición de equivalencia** – dividí los valores de entrada en grupos válidos y no válidos. Por ejemplo, probé imágenes de hasta 1 MB (válidas) e imágenes de más de 1 MB (no válidas).
- **Análisis de valores límite** – probé los valores en los límites. Por ejemplo, los campos de nombre se probaron con 55 caracteres (el máximo) y con 56 caracteres (uno por encima del máximo).
- **Pruebas de casos de uso** – probé los flujos completos que sigue un usuario: añadir, editar y eliminar un empleado.

6.2 Checklist

Primero, creé un checklist con 20 puntos. El checklist cubre la interfaz gráfica, los campos de cada sección, los campos obligatorios, las opciones de los desplegables y de los botones de opción, y el comportamiento de los botones Save y Delete. Algunos ejemplos:

- Verificar que la sección Add Employee contiene todos los campos de las especificaciones.
- Verificar que todos los campos obligatorios están marcados con *.
- Verificar que el desplegable Status contiene solo 2 opciones: Enabled y Disabled.
- Verificar que Male y Female son las únicas opciones del campo Gender.
- Verificar que la navegación entre campos funciona con la tecla Tab.

6.3 Casos de prueba

Después, escribí 44 casos de prueba. Los creé en JIRA como incidencias de tipo Test (VFP-5 a VFP-48) y añadí los pasos en Zephyr Squad. Cada caso de prueba contiene:

- un título que empieza por “Verify that…”;
- los pasos de la prueba;
- los datos de prueba, cuando son necesarios;
- el resultado esperado de cada paso;
- una prioridad (Highest, High, Medium, Low o Lowest);
- enlaces al epic VFP-1 y a la story VFP-2.

Asigné las prioridades según la importancia de cada función. Los flujos principales (añadir, editar, eliminar, campos obligatorios) recibieron la prioridad más alta. Las comprobaciones de formato y de límites de caracteres recibieron una prioridad más baja.

A continuación se muestra un ejemplo de caso de prueba con sus pasos.

VFP-5: Verificar los campos obligatorios de la sección Add Employee (Prioridad: Highest)

#	Paso de la prueba	Resultado esperado
1	Iniciar sesión en la aplicación	El usuario ha iniciado sesión en la aplicación
2	Ir a PIM >> Add Employee	Se abre la sección Add Employee
3	Rellenar todos los campos obligatorios, excepto First Name, y hacer clic en Save	El empleado no se guarda, el campo First Name es obligatorio
4	Rellenar todos los campos obligatorios, excepto Last Name, y hacer clic en Save	El empleado no se guarda, el campo Last Name es obligatorio
5	Hacer clic en la casilla Create Login Details	Se abre la sección Create Login Details
6	Rellenar todos los campos obligatorios, excepto User Name, y hacer clic en Save	El empleado no se guarda, el campo User Name es obligatorio
7	Rellenar todos los campos obligatorios, excepto Password, y hacer clic en Save	El empleado no se guarda, el campo Password es obligatorio
8	Rellenar todos los campos obligatorios, excepto Confirm Password, y hacer clic en Save	El empleado no se guarda, el campo Confirm Password es obligatorio
9	Rellenar todos los campos obligatorios, excepto Status, y hacer clic en Save	El empleado no se guarda, el campo Status es obligatorio

Los casos de prueba completos, con todos sus pasos, se pueden consultar en el repositorio del proyecto (Employee_test_cases_with_steps.pdf).

6.4 Checklist de pruebas API

Para la API de Employee, creé un checklist aparte. Cubre tres métodos: GET, POST y PUT. A continuación se muestran algunos ejemplos.

Método	Ejemplos de comprobaciones
GET /api/v1/employee/:id	Se devuelve una lista de empleados cuando existen; se devuelve un error de autorización con un token no válido; un id de empleado aleatorio devuelve un error.
POST /api/v1/employee/:id	Un empleado se guarda solo con los parámetros obligatorios; el empleado no se guarda sin el parámetro "firstName" o "lastName"; se devuelve un error con un tipo de dato no válido.
PUT /api/v1/employee/:id	Un empleado se actualiza con todos los parámetros aceptados; se devuelve un error con un nombre duplicado; se devuelve un error de autorización con un token no válido.

El checklist API completo se puede consultar en el repositorio del proyecto (API_test_checklist.csv).

7. Implementación de las pruebas

Antes de la fase de ejecución, preparé los siguientes elementos:

- Comprobé que el entorno de pruebas estaba en funcionamiento: https://opensource-demo.orangehrmlive.com/
- Verifiqué el acceso al entorno con la cuenta Admin / admin123.
- Creé un ciclo de pruebas en Zephyr, llamado “EMPLOYEE module”.
- Añadí los 44 casos de prueba al ciclo de pruebas.
- Creé una colección en Postman con las peticiones API.
- Generé un token de autorización para acceder a la API.

8. Ejecución de las pruebas

8.1 Ejecución de las pruebas funcionales

Ejecuté los casos de prueba del ciclo “EMPLOYEE module” entre el 18 y el 21 de mayo de 2022. En cada test, seguí los pasos y comparé el resultado real con el resultado esperado. Marqué cada paso como PASS o FAIL. Cuando un test fallaba, creaba un bug y lo vinculaba a la ejecución.

Se ejecutaron los 44 casos de prueba. 39 tests se superaron y 5 tests fallaron.

A continuación se muestran dos ejemplos de tests ejecutados: uno superado y uno fallido.

ID	Caso de prueba	Prioridad	Resultado
VFP-47	Verificar que se puede eliminar un empleado	Highest	PASS
VFP-37	Verificar que el campo Date of Birth no acepta fechas posteriores a la fecha actual	High	FAIL

VFP-47 (PASS): seleccioné un empleado en la Employee List e hice clic en Delete. El empleado se eliminó de la lista, como se esperaba.

VFP-37 (FAIL): introduje una fecha futura en el campo Date of Birth. El campo aceptó la fecha y no se mostró ningún error. Reporté el bug VFP-49 para este test.

Los resultados completos de la ejecución de los 44 tests se pueden consultar en el repositorio del proyecto (Employee_cycle_summary_execution.pdf).

8.2 Bugs reportados

Por cada test fallido, creé un bug en JIRA. En total, reporté 5 bugs: 2 con prioridad High y 3 con prioridad Low.

Bug	Título	Prioridad	Hallado por	Estado
VFP-49	El campo Date of Birth acepta una fecha posterior a la fecha actual	High	VFP-37	To Do
VFP-50	El campo License Expiry Date acepta una fecha anterior a la fecha actual	High	VFP-41	To Do
VFP-51	La sección Photograph acepta imágenes .jpg/.png/.gif de más de 1 MB	Low	VFP-19	To Do
VFP-52	Los campos First Name, Middle Name y Last Name no aceptan 55 caracteres, en Add Employee	Low	VFP-14	To Do
VFP-53	Los campos First Name, Middle Name y Last Name no aceptan 55 caracteres, en Edit Personal Details	Low	VFP-16	To Do

Cada informe de bug contiene los pasos para reproducirlo, el resultado esperado, el resultado real, la prioridad y los enlaces al test, al epic y a la story relacionados. A continuación se muestra un ejemplo.

Campo	VFP-49
Título	El campo Date of Birth acepta una fecha posterior a la fecha actual
Tipo / Prioridad	Bug / High
Informador / Asignado	Valentina Hatos
Pasos para reproducirlo	1. Iniciar sesión en la aplicación. 2. Ir a PIM >> Employee List. 3. Seleccionar un empleado y hacer clic en Edit. 4. Introducir en el campo Date of Birth una fecha posterior a la fecha actual.
Resultado esperado	El campo no se completa y se muestra un error.
Resultado real	El campo se completa.
Enlaces	VFP-37 (test), VFP-1 (epic), VFP-2 (story)

Los informes de bugs completos se pueden consultar en el repositorio del proyecto (Employee_created_bugs.pdf).

En los bugs VFP-52 y VFP-53, comprobé que los campos de nombre no aceptan más de 30 caracteres, mientras que las especificaciones exigen 55.

8.3 Pruebas API

Probé la API en Postman, basándome en el checklist API. Creé una colección llamada “Orange HRM API Collection”. La colección utiliza dos variables: {{base_url}} para la dirección de la aplicación y {{auth_token}} para la autorización.

Obtención del token de acceso

Primero, envié una petición POST a /index.php/oauth/issueToken. El body contiene el client id, el client secret y el grant type “client_credentials”. Añadí un script de test a esta petición. El script verifica que el código de estado es 200. Después guarda el token de la respuesta en la variable de entorno auth_token. Así, las demás peticiones utilizan el token automáticamente.

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
var res_body = pm.response.json();
pm.environment.set("auth_token", `Bearer ${res_body.access_token}`);
```

Peticiones de Employee

Para la API de Employee, configuré tres peticiones:

- **GET Employee** – devuelve los datos de un empleado según su id.
- **Save Employee (POST)** – guarda un empleado con los parámetros firstName y lastName.
- **Update Employee (PUT)** – actualiza los datos de un empleado, por ejemplo el género.

En cada petición, modifiqué los parámetros y el token según las comprobaciones del checklist API. Después verifiqué el código de estado y el body de la respuesta.

Otras peticiones de la colección

La colección también contiene peticiones para el módulo Time: Customers, Projects y Activities. Para cada uno, configuré peticiones GET, POST, PUT y DELETE. En DELETE Customers, añadí un script de test. El script verifica que el código de estado es 404 y que la respuesta contiene el mensaje “Customer Not Found”.

```
pm.test("Status code is 404", function () {
    pm.response.to.have.status(404);
});
pm.test("Verify the response body is correct", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.error.status).to.eql("404");
    pm.expect(jsonData.error.text).to.eql("Customer Not Found");
});
```

9. Cierre de las pruebas

En la última fase, recopilé los resultados finales y generé los informes finales.

- Se planificaron 44 casos de prueba y se ejecutaron todos.
- 39 tests se superaron y 5 tests fallaron.
- Se reportaron 5 bugs en total: 2 con prioridad High y 3 con prioridad Low.
- Generé la matriz de trazabilidad. Vincula la story VFP-2 con cada test, su ejecución y los bugs relacionados.
- Generé el gráfico de ejecución de pruebas del ciclo “EMPLOYEE module”.

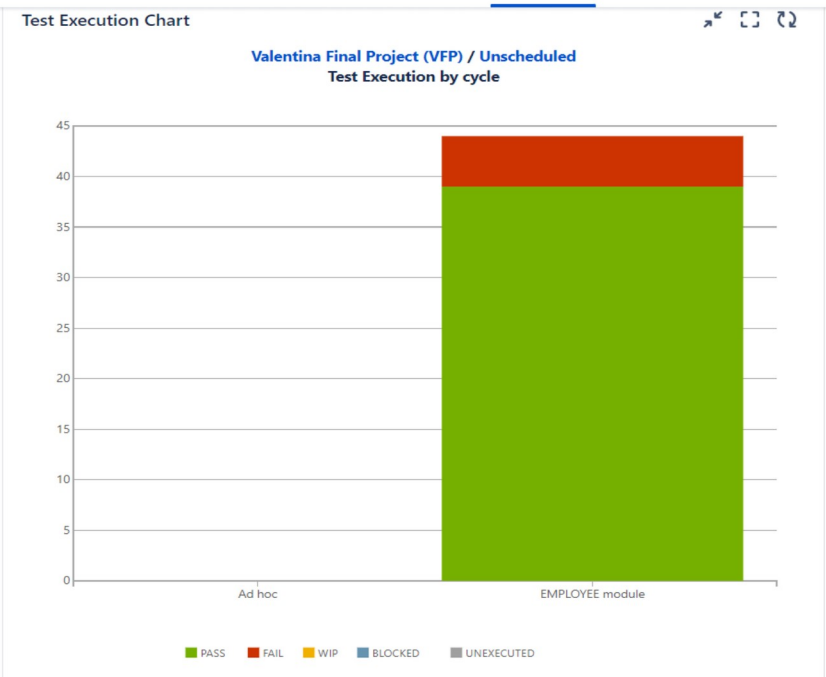


Figura 4: Gráfico final de ejecución de pruebas

9.1 Evaluación de los criterios de salida

Criterio de salida	Estado
Se han ejecutado todos los tests	Cumplido (44 de 44)
Se ha alcanzado la fecha límite	Cumplido
Los bugs sin resolver son insignificantes o de prioridad baja	Aún no cumplido – VFP-49 y VFP-50 tienen prioridad High
Todos los bugs resueltos se han vuelto a probar	Pendiente – los bugs aún no están corregidos
Regresión exploratoria en el módulo PIM	Planificada tras las correcciones

9.2 Conclusión

Las funciones principales del módulo Employee funcionan como se esperaba. Se pueden añadir, editar y eliminar empleados. Los campos obligatorios, los datos de acceso, los desplegables y la lista de empleados funcionan correctamente.

Todavía hay dos bugs de prioridad High abiertos. Están relacionados con la validación de las fechas en Date of Birth (VFP-49) y en License Expiry Date (VFP-50).

Recomendación: el equipo de pruebas recomienda el Go Live después de corregir y volver a probar VFP-49 y VFP-50. Los tres bugs de prioridad Low (VFP-51, VFP-52, VFP-53) se pueden corregir en una versión posterior. Tras las correcciones, es necesario un test de regresión completo en el módulo PIM.

10. Sección SQL

En la segunda parte del proyecto, utilicé MySQL Workbench. Creé una base de datos llamada “orangehrm” con dos tablas: departments y employees.

10.1 Creación de la base de datos y las tablas

```
create database orangehrm;
use orangehrm;
```

La tabla departments guarda el código, el nombre y el responsable de cada departamento. La columna department_code es la clave primaria.

```
create table departments(
department_code int primary key not null,
department_name varchar(55) not null,
department_manager varchar(55) not null
);
```

La tabla employees guarda los datos de los empleados. La columna employee_id es la clave primaria y aumenta automáticamente. La columna department es una clave foránea. Hace referencia a la columna department_code de la tabla departments.

```
create table employees(
employee_id int primary key auto_increment,
first_name varchar(55) not null,
last_name varchar(55) not null,
date_of_birth date,
job_title varchar(55),
department int not null,
constraint fk_departments_department_code foreign key (department)
  references departments(department_code)
);
```

Limité las columnas de nombre a 55 caracteres, el mismo límite que en las especificaciones.

10.2 Inserción de datos

Inserté 3 departamentos y 11 empleados.

Código	Departamento	Responsable
11	Human Resources	Janet Smith
12	Accounting	Mary Adams
13	IT	Michael Collins

```
insert into departments (department_code, department_name, department_manager)
values
(11, "Human Resources", "Janet Smith"), (12, "Accounting", "Mary Adams"),
(13, "IT", "Michael Collins");

insert into employees (first_name, last_name, date_of_birth, job_title,
department) values
("Katie", "Jones", "1983-08-20", "Financial administrator", 12),
("David", "Moore", "1976-07-19", "HR administrator", 11),
... (11 empleados en total)
```

Después comprobé el contenido de las dos tablas con consultas SELECT *.

10.3 Consultas

Escribí 15 consultas. Cubren filtrado, ordenación, agrupación, joins, subconsultas y la actualización y eliminación de datos. A continuación se muestran cuatro ejemplos.

Tarea 2: Seleccionar todos los datos de los empleados cuyo apellido es "Carter".

```
select * from employees where last_name = 'Carter';
```

Utilicé una condición WHERE para filtrar las filas por apellido. La consulta devuelve dos empleados.

Tarea 5: Seleccionar todos los datos de los empleados, incluidos los datos de su departamento.

```
select * from employees e inner join departments d
on e.department = d.department_code;
```

Uní las dos tablas mediante la clave foránea. Utilicé los alias "e" y "d" para que la consulta sea más corta.

Tarea 9: Seleccionar los nombres de los departamentos con más de tres empleados.

```
select d.department_name from departments d
where 3 < (select count(*) from employees where department = d.department_code);
```

Utilicé una subconsulta. Para cada departamento, la subconsulta cuenta sus empleados. Solo se devuelven los departamentos con más de tres empleados.

Tarea 14: Reasignar todos los empleados de Accounting (código 12) a Human Resources (código 11).

```
update employees set department = 11 where department = 12;
```

Utilicé una sentencia UPDATE con una condición WHERE. Solo se movieron los empleados de Accounting.

Las 15 tareas y sus consultas se pueden consultar en el repositorio del proyecto (orangehrm database.sql).

11. Reflexiones finales

Este proyecto me ayudó a aplicar el proceso de pruebas completo en una aplicación real. Planifiqué las pruebas, diseñé y ejecuté los casos de prueba, reporté bugs y generé informes en JIRA y Zephyr Squad. También practiqué las pruebas de API en Postman y escribí consultas SQL en MySQL Workbench.

Todos los archivos del proyecto (checklist, casos de prueba, ejecuciones, bugs, matriz de trazabilidad, colección de Postman y script SQL) están disponibles en mi repositorio de GitHub: Manual_Testing_Portofolio / Final Project.